ROFL(Runtime OFf-chain Logic) 상세 프로세스 가이드

개요

ROFL(Runtime OFf-chain Logic)은 Oasis Network에서 제공하는 오프체인 컴퓨팅 솔루션입니다. 블록체인 상에서 직접 실행하기에는 비용이 많이 들거나 복잡한 로직을 안전한 TEE(Trusted Execution Environment) 환경에서 실행할 수 있게 해주는 혁신적인 기술입니다.

1. ROFL의 핵심 개념

1.1 TEE(Trusted Execution Environment) 지원

Intel SGX (Software Guard Extensions)

- 하드웨어 기반 보안 기술
- 메모리 암호화 및 격리
- 원격 증명(Remote Attestation) 지원

Intel TDX (Trust Domain Extensions)

- 차세대 기밀 컴퓨팅 기술
- VM 레벨 격리 및 보안
- 더 큰 메모리 지원

1.2 앱 실행 환경

Raw 앱

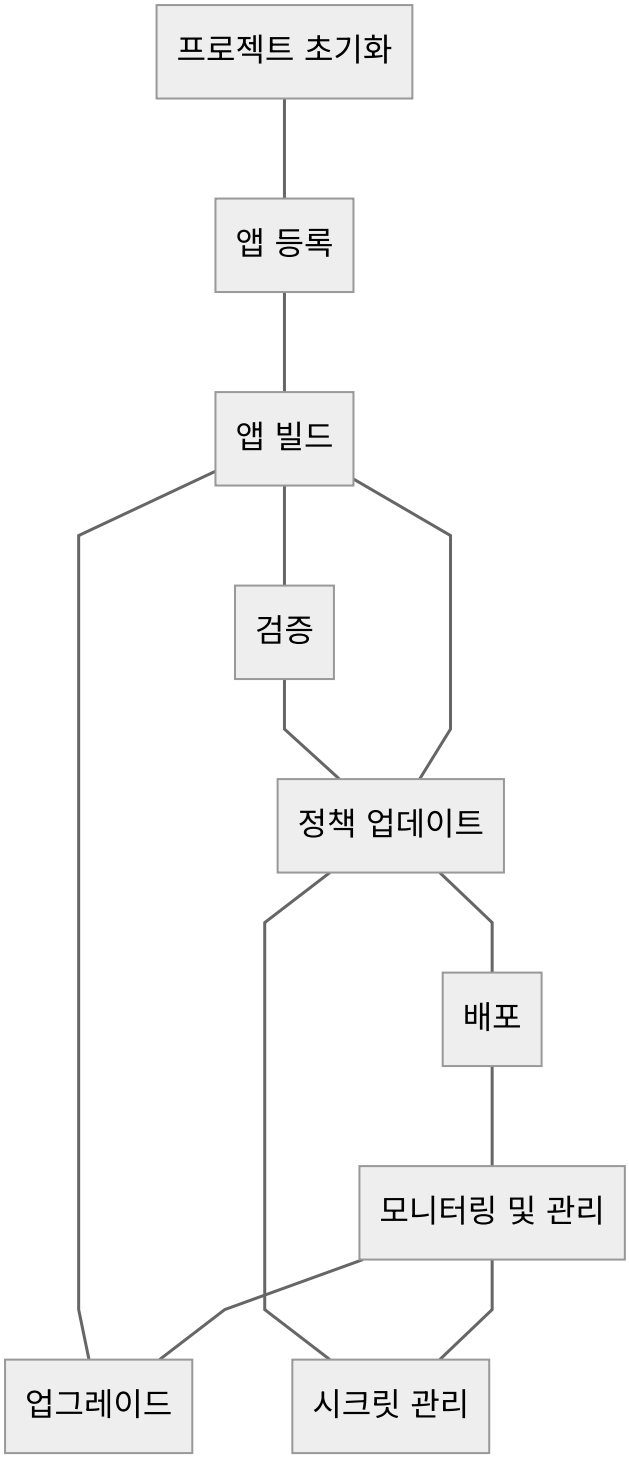
- 기본적인 바이너리 실행 환경
- 직접적인 시스템 접근
- 최적화된 성능

Container 앱

- Docker 컨테이너 기반 실행
- 복잡한 의존성 관리
- 개발 및 배포 편의성

2. ROFL 개발 생명주기

2.1 단계별 프로세스 개요



3. 상세 구현 프로세스

3.1 프로젝트 초기화

명령어 실행

```
oasis rofl init myapp --tee tdx --kind container
```

생성되는 파일 구조

```
myapp/
├── rofl.yaml # 매니페스트 파일
├── compose.yaml # Docker Compose 설정 (container 타입)
├── .gitignore # Git 무시 파일
└── .git/ # Git 저장소
```

매니페스트 파일 구조 (rofl.yaml)

```yaml
name: myapp
version: 0.1.0
tee: tdx
kind: container
resources:
memory: 512  # 메모리 (MB)
cpus: 1  # vCPU 개수
storage:
kind: disk-persistent
size: 512  # 스토리지 크기 (MB)


# 배포 설정 (자동 생성됨)
deployments:
```

```yaml
default:
network: ""  # 네트워크 이름
paratime: ""  # ParaTime 이름
admin: ""  # 관리자 계정
debug: false  # 디버그 모드
policy: # 보안 정책
quotes:
pcs:
tcb_validity_period: 30
min_tcb_evaluation_data_number: 18
tdx: {}
enclaves: [] # Enclave 식별자 목록
endorsements:
- any: {}
fees: 2  # 수수료 정책
max_expiration: 3
```

초기화 과정에서 수행되는 작업

1. **디렉토리 생성**: 앱 이름으로 디렉토리 생성
2. **매니페스트 생성**: 기본 설정으로 rofl.yaml 생성
3. **Git 초기화**: 새 Git 저장소 생성 (기존 저장소가 없는 경우)
4. **gitignore 설정**: *.orc 파일 제외 설정
5. **아티팩트 설정**: TEE 및 앱 종류에 따른 기본 아티팩트 구성

3.2 앱 등록 (Create)

명령어 실행

```
oasis rofl create --network testnet --account my_account --scheme cn
```

내부 프로세스 상세

1. 배포 설정 생성

```yaml
deployments:
default:
network: testnet
paratime: sapphire
admin: my_account
debug: true  # testnet의 경우 자동으로 true
policy:
quotes:
pcs:
tcb_validity_period: 30
min_tcb_evaluation_data_number: 18
tdx: {}
enclaves: [] # 초기에는 비어있음
endorsements:
- any: {} # 모든 endorsement 허용
fees: 2  # EndorsingNodePays
```

```
max_expiration: 3
trust_root:
height: 12345678
hash: "0xabcdef..."
```

2. 트랜잭션 생성

```
{
"method": "rofl.Create",
"body": {
"policy": {
"quotes": { /* 위의 정책 */ },
"enclaves": [],
"endorsements": [{"any": {}}],
"fees": 2,
"max_expiration": 3
},
"scheme": 1, // CreatorNonce
"metadata": {
"net.oasis.rofl.name": "myapp",
"net.oasis.rofl.version": "0.1.0"
}
}
```

```
}
```

3. 앱 ID 생성 및 저장

- 트랜잭션 실행 후 고유한 앱 ID 반환
- 매니페스트의 `deployments.default.app_id` 에 저장

출력 예시

```
Created ROFL app: rofl1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0
Run `oasis rofl build` to build your ROFL app.
```

3.3 앱 빌드

명령어 실행

```
oasis rofl build
```

빌드 환경 설정

환경 변수 설정

```
export ROFL_MANIFEST="rofl.yaml"
export ROFL_DEPLOYMENT_NAME="default"
export ROFL_DEPLOYMENT_NETWORK="testnet"
export ROFL_DEPLOYMENT_PARATIME="sapphire"
export ROFL_APP_ID="rofl1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0"
export ROFL_CONSENSUS_TRUST_ROOT="base64_encoded_trust_root"
```

```
export ROFL_TMPDIR="/tmp/oasis-build-xxx"
```

Trust Root 구성

```
{
"height": 12345678,
"hash": "0xabcdef...",
"runtime_id": "0000000000000000000000000000000000000000000000000a6d1e3ebf60dff6c
"chain_context": "0b91b8e4e44b2003a7c5e23ddadb5e14ef5345c0ebcb3ddcae07fa2f244ca
}
```

TEE별 빌드 프로세스

TDX Container 빌드

1. **컨테이너 환경 준비**

- Docker 빌더 이미지 사용
- 소스 코드를 `/src` 에 마운트
- 임시 빌드 디렉토리 생성

2. **아티팩트 다운로드**

```
artifacts:
firmware: "https://github.com/oasisprotocol/oasis-boot/releases/download/v0.6.2/
kernel: "https://github.com/oasisprotocol/oasis-boot/releases/download/v0.6.2/st
stage2: "https://github.com/oasisprotocol/oasis-boot/releases/download/v0.6.2/st
container:
```

```
runtime: "https://github.com/oasisprotocol/oasis-sdk/releases/download/rofl-cont
compose: "compose.yaml"
```

3. **번들 생성**

- ORC (Oasis Runtime Container) 파일 생성
- 모든 아티팩트를 단일 번들로 패키징
- 매니페스트 해시 계산

TDX Raw 빌드

1. **네이티브 바이너리 컴파일**
2. **기본 스테이지2 사용**
3. **최소한의 런타임 환경 구성**

SGX 빌드

1. **Intel SGX SDK 사용**
2. **Enclave 서명**
3. **원격 증명 설정**

Enclave 식별자 계산

```
# 빌드 완료 후 자동 실행
oasis rofl build
```

출력 예시

```
Building a ROFL application...
```

```
Deployment: default

Network: testnet

ParaTime: sapphire

Debug: true

App ID: rofl1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0

Name: myapp

Version: 0.1.0

TEE: tdx

Kind: container



ROFL app built and bundle written to 'myapp.default.orc'.

Computing enclave identity...



Update the manifest with the following identities to use the new app:



deployments:

default:

policy:

enclaves:

- "8f7e8d9c6b5a4f3e2d1c0b9a8f7e6d5c4b3a2f1e0d9c8b7a6f5e4d3c2b1a0f9e"
```

3.4 정책 업데이트

매니페스트 업데이트

빌드 후 생성된 Enclave 식별자를 매니페스트에 추가:

```
deployments:
default:
policy:
enclaves:
- id: "8f7e8d9c6b5a4f3e2d1c0b9a8f7e6d5c4b3a2f1e0d9c8b7a6f5e4d3c2b1a0f9e"
version: ""  # 최신 버전
description: "TDX container enclave"
```

온체인 업데이트

```
oasis rofl update --admin self
```

트랜잭션 내용

```
{
"method": "rofl.Update",
"body": {
"id": "rofl1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0",
"admin": "oasis1qp87hflmelnpqhzcqcw8rhzakq4elj7jzv090p3e",
"policy": {
"enclaves": ["8f7e8d9c6b5a4f3e2d1c0b9a8f7e6d5c4b3a2f1e0d9c8b7a6f5e4d3c2b1a0f9e"
/* 기타 정책 설정 */
},
```

```
"metadata": {
"net.oasis.rofl.name": "myapp",
"net.oasis.rofl.version": "0.1.0"
},
"secrets": {}
}
}
```

3.5 배포 (Deploy)

프로바이더 오퍼 조회

```
oasis rofl deploy --show-offers --provider oasis1qp0xk0q0q0q0q0q0q0q0q0q0q0q0
```

출력 예시

```
Offers available from the selected provider:

- basic-tdx [0x1234567890abcdef...]
TEE: tdx | Memory: 512 MiB | vCPUs: 1 | Storage: 1.00 GiB
Price: 0.1 TEST/hour | 50.0 TEST/month | 500.0 TEST/year

- high-performance [0xabcdef1234567890...]
TEE: tdx | Memory: 2048 MiB | vCPUs: 4 | Storage: 10.00 GiB
```

```
Price: 0.5 TEST/hour | 250.0 TEST/month | 2500.0 TEST/year
```

배포 실행

```
oasis rofl deploy --provider oasis1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q
--offer basic-tdx \
--term month \
--count 3
```

배포 프로세스 상세

1. OCI 저장소 푸시

```
# 내부적으로 실행됨
# ORC 번들을 OCI 형식으로 변환
# rofl.sh 또는 지정된 저장소에 업로드
```

OCI 매니페스트 구조

```
{
"schemaVersion": 2,
"mediaType": "application/vnd.oasis.orc",
"config": {
"mediaType": "application/vnd.oasis.orc.config.v1+json",
"digest": "sha256:...",
```

```
"size": 1234
},
"layers": [
{
"mediaType": "application/vnd.oasis.orc.layer.v1",
"digest": "sha256:...",
"size": 5678
}
],
"annotations": {
"org.opencontainers.image.created": "2025-03-31T00:00:00Z"
}
}
```

2. 머신 프로비저닝

```
{
"method": "roflmarket.InstanceCreate",
"body": {
"provider": "oasis1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0",
"offer": "0x1234567890abcdef...",
"deployment": {
"app_id": "rofl1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0",
"manifest_hash": "sha256:...",
```

```
"metadata": {
"net.oasis.deployment.orc.ref": "rofl.sh/myapp:1234567890@sha256:..."
}
},
"term": 2, // Month
"term_count": 3
}
}
```

3. 배포 스케줄링

```
{
"method": "roflmarket.InstanceExecuteCmds",
"body": {
"provider": "oasis1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0",
"id": "instance_id_12345",
"cmds": [
{
"method": "deploy",
"args": {
"deployment": { /* 배포 정보 */ },
"wipe_storage": false
}
```

```
}

]

}

}
```

출력 예시

```
Using provider: testnet-provider (oasis1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0

Pushing ROFL app to OCI repository 'rofl.sh/myapp:1234567890'...

Taking offer:

- basic-tdx [0x1234567890abcdef...]

TEE: tdx | Memory: 512 MiB | vCPUs: 1 | Storage: 1.00 GiB

Price: 0.1 TEST/hour | 50.0 TEST/month | 500.0 TEST/year



Selected per-month pricing term, total price is 150.0 TEST.

Created machine: instance_12345

Deployment into machine scheduled.

This machine expires on 2024-04-15 10:30:45. Use `oasis rofl machine top-up` to

Use `oasis rofl machine show` to check status.
```

4. 시크릿 관리

4.1 개별 시크릿 설정

파일에서 시크릿 읽기

```
oasis rofl secret set DATABASE_URL /path/to/db_url.txt --public-name "Dat
```

표준 입력에서 시크릿 읽기

```
echo "postgresql://user:pass@host:5432/db" | oasis rofl secret set DATABASE
```

4.2 환경변수 파일 일괄 가져오기

.env 파일 예시

```
# .env 파일
API_KEY=sk-1234567890abcdef
DATABASE_URL=postgresql://user:pass@localhost:5432/myapp
REDIS_URL=redis://localhost:6379/0
SECRET_KEY=super-secret-key-here

# 주석도 지원됩니다
DEBUG=true
```

일괄 가져오기

```
oasis rofl secret import .env
```

출력 예시

```
Imported 5 secrets, updated 0 existing from '.env'.

Run `oasis rofl update` to update your ROFL app's on-chain configuration.
```

4.3 시크릿 암호화 프로세스

1. SEK(Secret Encryption Key) 획득

```
# 내부적으로 실행됨

# 온체인에서 앱의 SEK 조회

app_config = conn.Runtime(paratime).ROFL.App(ctx, client.RoundLatest, app_id

sek = app_config.SEK
```

2. 클라이언트 측 암호화

```
// 내부 구현

func EncryptSecret(name string, value []byte, sek []byte) ([]byte, error) {

// X25519-Deoxys-II 암호화 사용

// 시크릿 이름과 값을 함께 암호화

return encrypt(name, value, sek)

}
```

3. 매니페스트 저장

```
deployments:
```

```
default:
secrets:
- name: "DATABASE_URL"
public_name: "Database Connection"
value: "base64_encrypted_value_here"
- name: "API_KEY"
value: "another_base64_encrypted_value"
```

4.4 시크릿 조회 및 관리

시크릿 정보 조회

```
oasis rofl secret get DATABASE_URL
```

출력 예시

```
Name: DATABASE_URL
Public name: Database Connection
Size: 45 bytes
```

시크릿 제거

```
oasis rofl secret rm API_KEY
```

5. 고급 기능

5.1 다중 배포 환경 관리

매니페스트 설정

```yaml
deployments:
testnet:
network: testnet
paratime: sapphire
admin: test_admin
debug: true
policy:
enclaves:
- "testnet_enclave_id_here"
mainnet:
network: mainnet
paratime: sapphire
admin: prod_admin
debug: false
policy:
enclaves:
- "mainnet_enclave_id_here"
quotes:
pcs:
```

```
tcb_validity_period: 90  # 더 엄격한 정책
```

특정 배포 환경 사용

```
# 테스트넷 배포
oasis rofl build --deployment testnet
oasis rofl deploy --deployment testnet

# 메인넷 배포
oasis rofl build --deployment mainnet
oasis rofl deploy --deployment mainnet
```

5.2 커스텀 아티팩트 사용

매니페스트 설정

```
artifacts:
builder: "myregistry/custom-builder:latest"
firmware: "https://mycdn.com/custom-firmware.fd#sha256hash"
kernel: "https://mycdn.com/custom-kernel.bin#sha256hash"
stage2: "https://mycdn.com/custom-stage2.tar.bz2#sha256hash"
container:
runtime: "https://mycdn.com/custom-runtime#sha256hash"
compose: "custom-compose.yaml"
```

아티팩트 업그레이드

```
# 최신 기본 아티팩트로 업그레이드
oasis rofl upgrade
```

5.3 빌드 스크립트 사용

매니페스트에 스크립트 정의

```
scripts:
build-pre: |
#!/bin/bash
echo "Pre-build script running..."
npm install
npm run build
build-post: |
#!/bin/bash
echo "Post-build script running..."
echo "Build completed at $(date)"
bundle-post: |
#!/bin/bash
echo "Bundle created: $ROFL_BUNDLE"
echo "Enclave ID: $ROFL_ENCLAVE_ID_0"
# 추가 검증 로직
```

5.4 검증 및 디버깅

빌드 검증

```
# 빌드된 Enclave ID와 매니페스트 비교
oasis rofl build --verify
```

검증 과정

1. 빌드된 Enclave 식별자 계산
2. 매니페스트의 최신 Enclave 식별자와 비교
3. 온체인 정책과 일치성 확인

오프라인 빌드

```
# 네트워크 접근 없이 빌드
oasis rofl build --offline
```

컨테이너 없이 빌드

```
# 네이티브 환경에서 빌드
oasis rofl build --no-container
```

검증만 수행

```
# 빌드 없이 매니페스트 검증만
oasis rofl build --only-validate
```

6. 모니터링 및 관리

6.1 앱 상태 조회

기본 정보 조회

```
oasis rofl show
```

출력 예시

```
App ID: rofl1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0

Admin: oasis1qp87hflmelnpqhzcqcw8rhzakq4elj7jzv090p3e

Staked amount: 0.0 TEST


Metadata:

net.oasis.rofl.name: myapp

net.oasis.rofl.version: 0.1.0


Secrets:

DATABASE_URL: [45 bytes]

API_KEY: [32 bytes]


Policy:

{

"quotes": {
```

"pcs": {

"tcb_validity_period": 30,

"min_tcb_evaluation_data_number": 18,

"tdx": {}

}

},

"enclaves": [

"8f7e8d9c6b5a4f3e2d1c0b9a8f7e6d5c4b3a2f1e0d9c8b7a6f5e4d3c2b1a0f9e"

],

"endorsements": [

{"any": {}}

],

"fees": 2,

"max_expiration": 3

}

=== Replicas ===

- RAK: ed25519:AbCdEf1234567890...

Node ID: node1234567890abcdef...

Expiration: 12345

Metadata:

provider: testnet-provider

```
machine: instance_12345
```

6.2 머신 관리

머신 상태 조회

```
oasis rofl machine show --machine default
```

출력 예시

```
Machine ID: instance_12345

Provider: oasis1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0

Offer: basic-tdx

Status: running

Created: 2024-01-15T10:30:45Z

Expires: 2024-04-15T10:30:45Z

Resources:

Memory: 512 MiB

vCPUs: 1

Storage: 1.00 GiB

Deployment:

App ID: rofl1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0

Status: deployed

Last Update: 2024-01-15T11:00:00Z
```

머신 로그 조회

```
oasis rofl machine logs --machine default --tail 100
```

머신 연장

```
# 3개월 연장
oasis rofl machine top-up --machine default --term month --count 3
```

트랜잭션 예시

```
{
"method": "roflmarket.InstanceTopUp",
"body": {
"provider": "oasis1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0",
"id": "instance_12345",
"term": 2, // Month
"term_count": 3
}
}
```

6.3 생명주기 관리

앱 업그레이드

```
# 1. 아티팩트 업그레이드
```

```
oasis rofl upgrade


# 2. 새 버전 빌드
oasis rofl build


# 3. 정책 업데이트
oasis rofl update


# 4. 재배포 (필요한 경우)
oasis rofl deploy --force
```

앱 완전 제거

```
oasis rofl remove
```

경고 메시지

```
WARNING: Removing this ROFL app will DEREGISTER it, ERASE any on-chain secrets a
WARNING: THIS ACTION IS IRREVERSIBLE!
Remove ROFL app 'rofl1qp0xk0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0q0' deployed
```

7. 실제 사용 시나리오

7.1 시나리오 1: REST API 서비스

프로젝트 구조

```
api-service/
├── rofl.yaml
├── compose.yaml
├── src/
| ├── main.py
| ├── requirements.txt
| └── config.py
└── Dockerfile
```

Docker Compose 설정

```yaml
# compose.yaml
version: '3.8'
services:
api:
build: .
ports:
- "8080:8080"
environment:
- DATABASE_URL=${DATABASE_URL}
- API_SECRET=${API_SECRET}
- REDIS_URL=${REDIS_URL}
```

```
volumes:
- app_data:/app/data
restart: unless-stopped

volumes:
app_data:
```

배포 과정

```
# 1. 프로젝트 초기화
oasis rofl init api-service --kind container

# 2. 앱 등록
oasis rofl create --network testnet --account api_admin

# 3. 시크릿 설정
echo "postgresql://user:pass@db:5432/api" | oasis rofl secret set DATABASE_
echo "super-secret-api-key" | oasis rofl secret set API_SECRET -
echo "redis://redis:6379/0" | oasis rofl secret set REDIS_URL -

# 4. 빌드 및 배포
oasis rofl build
oasis rofl update
```

```
oasis rofl deploy --offer high-performance
```

7.2 시나리오 2: 데이터 처리 배치 작업

Raw 앱 구조

```
data-processor/
├── rofl.yaml
├── src/
| ├── main.go
| ├── processor.go
| └── go.mod
└── Makefile
```

매니페스트 설정

```yaml
name: data-processor
version: 1.0.0
tee: sgx
kind: raw
resources:
memory: 2048
cpus: 4
storage:
kind: disk-ephemeral
```

```
size: 10240


scripts:
build-pre: |
#!/bin/bash
cd src
go mod download
build-post: |
#!/bin/bash
cd src
CGO_ENABLED=0 GOOS=linux go build -o ../processor .
```

배포 과정

```
# 1. SGX Raw 앱으로 초기화
oasis rofl init data-processor --tee sgx --kind raw


# 2. 고성능 머신에 배포
oasis rofl create --network mainnet --account batch_admin
oasis rofl build
oasis rofl update
oasis rofl deploy --offer high-memory --term year --count 1
```

7.3 시나리오 3: 마이크로서비스 아키텍처

복합 서비스 구조

```
microservices/
├── rofl.yaml
├── compose.yaml
├── auth-service/
| ├── Dockerfile
| └── src/
├── user-service/
| ├── Dockerfile
| └── src/
├── notification-service/
| ├── Dockerfile
| └── src/
└── nginx/
└── nginx.conf
```

Docker Compose 설정

```yaml
# compose.yaml
version: '3.8'
services:
nginx:
```

```
    image: nginx:alpine
    ports:
    - "80:80"
    - "443:443"
    volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf
    depends_on:
    - auth-service
    - user-service
    - notification-service


    auth-service:
    build: ./auth-service
    environment:
    - JWT_SECRET=${JWT_SECRET}
    - DATABASE_URL=${AUTH_DB_URL}
    user-service:
    build: ./user-service
    environment:
    - DATABASE_URL=${USER_DB_URL}
    - REDIS_URL=${REDIS_URL}
    notification-service:
    build: ./notification-service
```

```
    environment:
    - SMTP_URL=${SMTP_URL}
    - QUEUE_URL=${QUEUE_URL}

  redis:
    image: redis:alpine
    volumes:
    - redis_data:/data

volumes:
  redis_data:
```

환경별 배포

```
# 개발 환경
oasis rofl build --deployment development
oasis rofl deploy --deployment development --offer basic-tdx

# 스테이징 환경
oasis rofl build --deployment staging
oasis rofl deploy --deployment staging --offer standard-tdx

# 프로덕션 환경
```

```
oasis rofl build --deployment production

oasis rofl deploy --deployment production --offer enterprise-tdx
```

8. 문제 해결 및 디버깅

8.1 일반적인 문제들

빌드 실패

```
# 상세 로그와 함께 빌드
oasis rofl build --verbose

# 컨테이너 없이 네이티브 빌드 시도
oasis rofl build --no-container

# 오프라인 모드로 빌드 (네트워크 문제 시)
oasis rofl build --offline
```

Enclave ID 불일치

```
# 강제로 매니페스트 업데이트
oasis rofl build --force

# 검증 모드로 차이점 확인
```

```
oasis rofl build --verify
```

배포 실패

```
# 기존 배포 강제 덮어쓰기
oasis rofl deploy --force

# 만료된 머신 교체
oasis rofl deploy --replace-machine
```

8.2 로그 및 모니터링

상세 로그 확인

```
# 환경 변수로 로그 레벨 설정
OASIS_LOG_LEVEL=debug oasis rofl build

# 특정 모듈만 디버그
OASIS_LOG_LEVEL=info,rofl=debug oasis rofl deploy
```

머신 상태 모니터링

```
# 실시간 로그 모니터링
oasis rofl machine logs --machine default --follow
```

```
# 머신 리소스 사용량 확인
oasis rofl machine show --machine default --verbose
```

8.3 복구 절차

매니페스트 복구

```
# 백업에서 복구
cp rofl.yaml.backup rofl.yaml

# 온체인 상태에서 매니페스트 재생성
oasis rofl show > current_state.json
# 수동으로 매니페스트 재구성
```

머신 복구

```
# 새 머신으로 재배포
oasis rofl deploy --replace-machine

# 기존 머신 연장
oasis rofl machine top-up --machine default --term month --count 1
```

9. 보안 고려사항

9.1 TEE 보안

SGX 보안 모델

- 하드웨어 기반 메모리 암호화
- 원격 증명을 통한 코드 무결성 검증
- Side-channel 공격 방어

TDX 보안 모델

- VM 레벨 격리
- 메모리 암호화 및 무결성 보호
- 하이퍼바이저로부터의 보호

9.2 시크릿 관리 보안

암호화 방식

- X25519-Deoxys-II 암호화 사용
- 앱별 고유 SEK(Secret Encryption Key)
- 클라이언트 측 암호화로 서버에 평문 노출 방지

접근 제어

- 관리자 계정을 통한 접근 제어
- Enclave 식별자 기반 인증
- 만료 시간 설정을 통한 시간 제한

9.3 네트워크 보안

통신 암호화

- TLS를 통한 모든 네트워크 통신 암호화
- gRPC 기반 안전한 API 통신
- 트랜잭션 서명을 통한 무결성 보장

10. 성능 최적화

10.1 리소스 최적화

메모리 사용량 최적화

```
resources:
memory: 512  # 필요한 최소 메모리만 할당
cpus: 1  # CPU 집약적 작업이 아니라면 1개로 충분
storage:
kind: ram  # 임시 데이터는 RAM 스토리지 사용
size: 256
```

스토리지 최적화

```
resources:
storage:
kind: disk-ephemeral  # 영구 저장이 불필요하면 ephemeral 사용
size: 1024  # 필요한 최소 크기만 할당
```

10.2 빌드 최적화

컨테이너 이미지 최적화

```
# 멀티 스테이지 빌드 사용
FROM node:18-alpine AS builder
```

```
WORKDIR /app

COPY package*.json ./

RUN npm ci --only=production



FROM node:18-alpine

WORKDIR /app

COPY --from=builder /app/node_modules ./node_modules

COPY . .

EXPOSE 3000

CMD ["node", "server.js"]
```

빌드 캐시 활용

```
# 빌드 아티팩트 캐시 활용

oasis  rofl  build  --no-container  # 로컬 캐시 사용
```

이 문서는 ROFL의 전체적인 개발 및 배포 프로세스를 상세히 다루고 있으며, 실제 프로덕션 환경에서 ROFL 앱을 성공적으로 구축하고 운영하는 데 필요한 모든 정보를 제공합니다.